

# Pitch to Postgres: Designing and Querying a Relational Database for FIFA World Cup Data

DS604 Introduction to Data Management  
Milestone 5: Views

| Group member      | Student ID |
|-------------------|------------|
| Devanshi Dudhatra | 202618027  |
| Pujita Sunapu     | 202618003  |
| Rahul Saha        | 202618037  |

Due 6 November 2026

Data: The Fjelstul World Cup Database by Joshua C. Fjelstul (CC-BY-SA 4.0)

## 1. The seven views

`sql/views.sql` defines seven views, built from the strongest Milestone 4 queries. A view stores no data: it is a named `SELECT` that PostgreSQL runs against the current tables every time it is queried. The executed notebook `notebooks/05_views.ipynb` (with `05_views.pdf`) shows `SELECT * ... LIMIT 10` and an example query for each view.

| View                              | Rows  | One row per           | Built from             | What it gives the user                                                                                                                                             |
|-----------------------------------|-------|-----------------------|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>v_match_results</code>      | 900   | match                 | joins of Q07, Q12, Q18 | Readable match list with team names, score text (with shoot-out score), result, winner including shoot-outs, and whether the match went to extra time or penalties |
| <code>v_top_scorers</code>        | 1,297 | goal scorer           | Q10                    | All-time ranking counted <b>per person</b> , so goals for West Germany and Germany are combined                                                                    |
| <code>v_tournament_summary</code> | 21    | tournament            | Q01, Q11, Q17, Q23     | Fact sheet: hosts, format (stage sequence), teams, final four, whether the host won, goals, top scorer(s), extra time, shoot-outs, cards                           |
| <code>v_team_performance</code>   | 84    | team                  | Q01, Q02               | Participation span, W/D/L, goals, win %, titles, finals, top-four finishes, best finish                                                                            |
| <code>v_referee_discipline</code> | 380   | referee               | Q22                    | Matches, tournaments, and, since 1970, cards, sendings-off and cards per match                                                                                     |
| <code>v_host_performance</code>   | 22    | host per tournament   | Q03                    | Each host's record and finish in its own tournament                                                                                                                |
| <code>v_head_to_head</code>       | 1,230 | (team, opponent) pair | Q07                    | Any head-to-head in one <code>WHERE</code> clause                                                                                                                  |

`v_match_results` and `v_tournament_summary` also rebuild the readable columns that the normalised schema deliberately does not store (Milestone 2, Section 3.2): score text, result labels, hosts, champion and format flags.

## 2. Testing (local PostgreSQL 18)

| Test                                                                             | Result                                                                                                                                                                                                                                        |
|----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>views.sql</code> runs without errors, and running it again changes nothing | Yes ( <code>CREATE OR REPLACE VIEW</code> , no <code>DROP</code> )                                                                                                                                                                            |
| Row counts as expected                                                           | 900 / 1,297 / 21 / 84 / 380 / 22 / 1,230                                                                                                                                                                                                      |
| Each view agrees with the query it was built from                                | 8 / 8 checks pass. Examples: <code>v_team_performance</code> = Q02 for all 49 teams, <code>v_referee_discipline</code> = Q22, <code>v_tournament_summary</code> goals = Q17 for every year, <code>v_head_to_head</code> Brazil v Sweden = Q07 |

**A bug found by testing the views.** Comparing `v_top_scorers` with Q10 exposed an error in three Milestone 4 queries (Q10, Q26, Q27). `FETCH FIRST 10 ROWS WITH TIES` only keeps rows tied on the *entire* `ORDER BY`, and we had added a name as a tie-breaker, so tied rows were cut off. Q10 had dropped three players who also scored 10 goals (Helmut Rahn, Teófilo Cubillas, Thomas Müller), and Q26 had dropped eight stadiums with 9 matches. The queries now rank in a derived table and filter on `rank <= 10`, which keeps all ties. The Milestone 4 notebook was re-run with the corrected queries.

**A practical note.** `CREATE OR REPLACE VIEW` can add columns to a view but cannot change an existing column's data type. If a view definition is later changed in that way, the old view must be dropped first. That needs the right

permissions on the course server, so agree it in the group first.

### 3. Why views, and when a table would be better

---

- **One definition, many users.** Analysts, broadcasters and the final report can query `v_team_performance` instead of copying a 40-line query, and everyone gets the same conventions (for example, a shoot-out counts as a draw).
- **Normalised storage, readable results.** Facts are stored once in the tables; the views present them joined and labelled.
- **Never out of date.** A view is recomputed on every query. A summary *table* would have to be rebuilt after every data change, and could silently disagree with the tables if someone forgot.
- **When a table is better.** If a summary were expensive and read very often on a large database, a `MATERIALIZED VIEW` (results stored and refreshed on demand) or a summary table would be faster. With 57,269 rows every view here answers in milliseconds, so plain views are the right choice.

### 4. View definitions (`sql/views.sql`)

---

```
-- =====
-- Pitch to Postgres: Designing and Querying a Relational Database for FIFA World Cup Data
-- DS604 Introduction to Data Management - Milestone 5: Views
-- Group: Devanshi Dudhatra (202618027), Pujita Sunapu (202618003), Rahul Saha (202618037)
--
-- Data: Joshua C. Fjelstul, The Fjelstul World Cup Database, https://github.com/jfjelstul/worldcup,
--       CC-BY-SA 4.0. Coverage: the 21 men's World Cups 1930-2018.
--
-- Seven views built from the strongest queries of Milestone 4. Views store no data: each is a saved
-- SELECT that runs against the current tables whenever it is queried, so it is always up to date.
-- Conventions are the same as in sql/queries.sql (results come from the score, so a shoot-out is a
-- draw; historical teams are separate teams).
-- Safe to run more than once and on the course server: CREATE OR REPLACE VIEW, no DROP or DELETE.
-- Views have no ORDER BY; ordering is chosen by whoever queries them.
-- =====

-- =====
-- V1. v_match_results - every match with names and result labels
-- Purpose: One readable row per match: tournament, stage, venue, team names, score text, result,
--       winner (including shoot-out winners) and how far the match went. It rebuilds the derived columns
--       (score text, result labels) that the normalised schema does not store.
-- Based on: matches, with the joins used in Q07, Q12, Q18
-- SQL concepts: 6-table JOIN (teams joined twice under different aliases), CASE, string
-- concatenation
-- Expected result: 900 rows (one per match).
-- =====

CREATE OR REPLACE VIEW v_match_results AS
SELECT m.match_id,
       tr.year,
       st.stage_number,
       st.stage_name,
       m.group_name,
       m.match_date,
       m.match_time,
       s.stadium_name,
       s.city_name,
```

```

s.country_name AS stadium_country,
h.team_name AS home_team,
a.team_name AS away_team,
m.home_team_score,
m.away_team_score,
CAST(m.home_team_score AS VARCHAR(2)) || '-' || CAST(m.away_team_score AS VARCHAR(2))
|| CASE WHEN m.penalty_shootout
THEN ' (' || CAST(m.home_team_score_penalties AS VARCHAR(2)) || '-'
|| CAST(m.away_team_score_penalties AS VARCHAR(2)) || ' pens)'
ELSE '' END AS score,
CASE WHEN m.home_team_score > m.away_team_score THEN 'home win'
WHEN m.home_team_score < m.away_team_score THEN 'away win'
ELSE 'draw' END AS result,
CASE WHEN m.home_team_score > m.away_team_score
OR m.home_team_score_penalties > m.away_team_score_penalties THEN h.team_name
WHEN m.home_team_score < m.away_team_score
OR m.home_team_score_penalties < m.away_team_score_penalties THEN a.team_name
END AS winner_incl_shootout, --
NULL = drawn, no shoot-out
CASE WHEN m.penalty_shootout THEN 'penalties'
WHEN m.extra_time THEN 'extra time'
ELSE '90 minutes' END AS went_to,
m.replayed,
m.replay
FROM matches m
JOIN tournaments tr ON tr.tournament_id = m.tournament_id
JOIN tournament_stages st ON st.tournament_id = m.tournament_id AND st.stage_number = m.stage_number
JOIN stadiums s ON s.stadium_id = m.stadium_id
JOIN teams h ON h.team_id = m.home_team_id
JOIN teams a ON a.team_id = m.away_team_id;

-- =====
-- V2. v_top_scorers - all-time goal scorers
-- Purpose: One row per player who scored at least one goal (own goals excluded), with an all-time
-- rank. Counted per PERSON, so a player who played for two teams (e.g. West Germany and Germany)
-- gets one combined row.
-- Based on: Q10 (all-time top scorers)
-- SQL concepts: JOIN, WHERE, GROUP BY, window function RANK(), STRING_AGG(DISTINCT ...)
-- Expected result: 1,297 rows (one per scorer); goal_rank 1 = Miroslav Klose (16).
-- =====
CREATE OR REPLACE VIEW v_top_scorers AS
SELECT RANK() OVER (ORDER BY COUNT(*) DESC) AS goal_rank,
p.player_id,
COALESCE(p.given_name || ' ', '') || p.family_name AS player,
STRING_AGG(DISTINCT t.team_name, ' / ') AS teams,
COUNT(*) AS goals,
SUM(CASE WHEN g.penalty THEN 1 ELSE 0 END) AS penalty_goals,
COUNT(DISTINCT g.tournament_id) AS tournaments_scored_in,
COUNT(DISTINCT g.match_id) AS matches_scored_in,
MIN(tr.year) AS first_goal_year,
MAX(tr.year) AS last_goal_year
FROM goals g
JOIN players p ON p.player_id = g.player_id
JOIN teams t ON t.team_id = g.player_team_id
JOIN tournaments tr ON tr.tournament_id = g.tournament_id
WHERE NOT g.own_goal
GROUP BY p.player_id, p.given_name, p.family_name;

```

```

-- =====
-- V3. v_tournament_summary - one row per World Cup
-- Purpose: A fact sheet for each tournament: hosts, format, number of teams and matches, the final
-- four, goals, top scorer(s), cards and shoot-outs. It rebuilds the host, winner and format
-- information that the normalised tournaments table does not store.
-- Based on: Q01 (standings), Q11 (top scorer), Q17 (goals per match), Q23 (cards)
-- SQL concepts: several CTEs, window function RANK(), LEFT JOINs (no cards before 1970 gives NULL,
-- not 0), correlated scalar subqueries, STRING_AGG with ORDER BY, EXISTS
-- Expected result: 21 rows (1930-2018).
-- =====
CREATE OR REPLACE VIEW v_tournament_summary AS
WITH match_stats AS (
    SELECT tournament_id,
           COUNT(*) AS matches,
           SUM(home_team_score + away_team_score) AS goals,
           SUM(CASE WHEN extra_time THEN 1 ELSE 0 END) AS extra_time_matches,
           SUM(CASE WHEN penalty_shootout THEN 1 ELSE 0 END) AS shootouts
    FROM matches
    GROUP BY tournament_id
),
card_stats AS (
    SELECT tournament_id,
           COUNT(*) AS cards,
           SUM(CASE WHEN red_card OR second_yellow_card THEN 1 ELSE 0 END) AS sendings_off
    FROM bookings
    GROUP BY tournament_id
),
scorer_tally AS (
    SELECT tournament_id, player_id, COUNT(*) AS goals,
           RANK() OVER (PARTITION BY tournament_id ORDER BY COUNT(*) DESC) AS rnk
    FROM goals
    WHERE NOT own_goal
    GROUP BY tournament_id, player_id
),
top_scorers AS (
    SELECT st.tournament_id,
           STRING_AGG(COALESCE(p.given_name || ' ', '') || p.family_name, ', ' ORDER BY
p.family_name) AS top_scorers,
           MAX(st.goals) AS top_scorer_goals
    FROM scorer_tally st
    JOIN players p ON p.player_id = st.player_id
    WHERE st.rnk = 1
    GROUP BY st.tournament_id
)
SELECT tr.year,
       tr.tournament_name,
       tr.start_date,
       tr.end_date,
       (SELECT STRING_AGG(t.team_name, ' & ' ORDER BY t.team_name)
        FROM host_countries h JOIN teams t ON t.team_id = h.team_id
        WHERE h.tournament_id = tr.tournament_id) AS hosts,
       (SELECT STRING_AGG(s.stage_name, ' > ' ORDER BY s.stage_number)
        FROM tournament_stages s WHERE s.tournament_id = tr.tournament_id) AS format,
       (SELECT COUNT(*) FROM qualified_teams q WHERE q.tournament_id = tr.tournament_id) AS teams,
       ms.matches,
       (SELECT t.team_name FROM tournament_standings s JOIN teams t ON t.team_id = s.team_id

```

```

        WHERE s.tournament_id = tr.tournament_id AND s.position = 1)          AS champion,
    (SELECT t.team_name FROM tournament_standings s JOIN teams t ON t.team_id = s.team_id
     WHERE s.tournament_id = tr.tournament_id AND s.position = 2)          AS runner_up,
    (SELECT t.team_name FROM tournament_standings s JOIN teams t ON t.team_id = s.team_id
     WHERE s.tournament_id = tr.tournament_id AND s.position = 3)          AS third_place,
    (SELECT t.team_name FROM tournament_standings s JOIN teams t ON t.team_id = s.team_id
     WHERE s.tournament_id = tr.tournament_id AND s.position = 4)          AS fourth_place,
    CASE WHEN EXISTS (SELECT 1 FROM host_countries h
                     JOIN tournament_standings s ON s.tournament_id = h.tournament_id AND
s.team_id = h.team_id
                     WHERE h.tournament_id = tr.tournament_id AND s.position = 1)
    THEN TRUE ELSE FALSE END                                             AS host_won,
    ms.goals,
    ROUND(1.0 * ms.goals / ms.matches, 2)                               AS goals_per_match,
    ts.top_scorers,
    ts.top_scorer_goals,
    ms.extra_time_matches,
    ms.shootouts,
    cs.cards,                                                            -- NULL before 1970
(no cards)
    cs.sendings_off,
    ROUND(1.0 * cs.cards / ms.matches, 2)                               AS cards_per_match
FROM tournaments tr
JOIN match_stats ms      ON ms.tournament_id = tr.tournament_id
LEFT JOIN card_stats cs  ON cs.tournament_id = tr.tournament_id
LEFT JOIN top_scorers ts ON ts.tournament_id = tr.tournament_id;

-- =====
-- V4. v_team_performance - all-time summary for every team
-- Purpose: One row per national team: participation span, match record (shoot-out = draw), goals,
-- win percentage, titles, finals, top-four finishes and best finish.
-- Based on: Q01 (titles, finals, top four) and Q02 (all-time records)
-- SQL concepts: three aggregating CTEs combined with JOIN and LEFT JOIN, COALESCE for teams that
never
-- reached the top four, CASE to label the best finish
-- Expected result: 84 rows (one per team); Brazil: 109 matches, 73 won, 5 titles.
-- =====
CREATE OR REPLACE VIEW v_team_performance AS
WITH record AS (
    SELECT team_id,
           COUNT(*)                               AS matches,
           SUM(CASE WHEN goals_for > goals_against THEN 1 ELSE 0 END) AS won,
           SUM(CASE WHEN goals_for = goals_against THEN 1 ELSE 0 END) AS drawn,
           SUM(CASE WHEN goals_for < goals_against THEN 1 ELSE 0 END) AS lost,
           SUM(goals_for)                                             AS goals_for,
           SUM(goals_against)                                         AS goals_against
    FROM team_appearances
    GROUP BY team_id
),
participation AS (
    SELECT q.team_id,
           COUNT(*)          AS tournaments,
           MIN(tr.year) AS first_year,
           MAX(tr.year) AS last_year
    FROM qualified_teams q
    JOIN tournaments tr ON tr.tournament_id = q.tournament_id
    GROUP BY q.team_id

```

```

),
finishes AS (
    SELECT team_id,
           SUM(CASE WHEN position = 1 THEN 1 ELSE 0 END) AS titles,
           SUM(CASE WHEN position <= 2 THEN 1 ELSE 0 END) AS finals,
           COUNT(*) AS top_four,
           MIN(position) AS best_position
    FROM tournament_standings
    GROUP BY team_id
)
SELECT t.team_id,
       t.team_name,
       t.team_code,
       c.confederation_code,
       p.tournaments,
       p.first_year,
       p.last_year,
       r.matches, r.won, r.drawn, r.lost,
       r.goals_for, r.goals_against,
       r.goals_for - r.goals_against AS goal_diff,
       ROUND(100.0 * r.won / r.matches, 1) AS win_pct,
       COALESCE(f.titles, 0) AS titles,
       COALESCE(f.final, 0) AS final,
       COALESCE(f.top_four, 0) AS top_four,
       CASE f.best_position WHEN 1 THEN 'champion' WHEN 2 THEN 'runner-up'
                           WHEN 3 THEN 'third' WHEN 4 THEN 'fourth'
                           ELSE 'outside top four' END AS best_finish
FROM teams t
JOIN confederations c ON c.confederation_id = t.confederation_id
JOIN participation p ON p.team_id = t.team_id
JOIN record r ON r.team_id = t.team_id
LEFT JOIN finishes f ON f.team_id = t.team_id;

-- =====
-- V5. v_referee_discipline - cards shown by each referee
-- Purpose: One row per referee: matches refereed, tournaments, and (for matches since 1970, when
cards
-- were introduced) cards, sendings-off and cards per match.
-- Based on: Q22 (strictest referees)
-- SQL concepts: CTE with LEFT JOIN (card-free matches count 0), GROUP BY, conditional aggregation,
-- NULLIF to avoid division by zero (referees who worked only before 1970 get NULL)
-- Expected result: 380 rows (one per referee); with at least 5 matches since 1970, the highest
-- cards_per_match is Lubos Michel (8.00), as in Q22.
-- =====
CREATE OR REPLACE VIEW v_referee_discipline AS
WITH per_match AS (
    SELECT ra.referee_id, ra.match_id, tr.year,
           COUNT(b.booking_id) AS cards,
           SUM(CASE WHEN b.red_card OR b.second_yellow_card THEN 1 ELSE 0 END) AS sendings_off
    FROM referee_appearances ra
    JOIN tournaments tr ON tr.tournament_id = ra.tournament_id
    LEFT JOIN bookings b ON b.match_id = ra.match_id
    GROUP BY ra.referee_id, ra.match_id, tr.year
)
SELECT r.referee_id,
       COALESCE(r.given_name || ' ', '') || r.family_name AS referee,
       r.country_name AS country,

```

```

        c.confederation_code,
        COUNT(*)                                AS matches,
        COUNT(DISTINCT pm.year)                 AS tournaments,
        MIN(pm.year)                           AS first_year,
        MAX(pm.year)                           AS last_year,
        SUM(CASE WHEN pm.year >= 1970 THEN 1 ELSE 0 END) AS matches_since_1970,
        CAST(SUM(pm.cards) AS INTEGER)          AS cards,
        CAST(COALESCE(SUM(pm.sendings_off), 0) AS INTEGER) AS sendings_off,
        ROUND(1.0 * SUM(pm.cards) / NULLIF(SUM(CASE WHEN pm.year >= 1970 THEN 1 ELSE 0 END), 0), 2)
                                                AS cards_per_match

FROM per_match pm
JOIN referees r      ON r.referee_id = pm.referee_id
JOIN confederations c ON c.confederation_id = r.confederation_id
GROUP BY r.referee_id, r.given_name, r.family_name, r.country_name, c.confederation_code;

-- =====
-- V6. v_host_performance - how each host nation did
-- Purpose: One row per host nation per tournament: matches, wins, draws, losses, goals, stage
reached,
-- final position and whether it won the title. Measures host advantage tournament by tournament.
-- Based on: Q03 (host-country advantage)
-- SQL concepts: JOIN through the composite key (tournament_id, team_id), LEFT JOIN to standings
-- (most hosts did not finish in the top four), aggregating subquery in FROM, CASE
-- Expected result: 22 rows (21 tournaments; 2002 had two hosts); 6 hosts won the title.
-- =====
CREATE OR REPLACE VIEW v_host_performance AS
SELECT tr.year,
       t.team_name                                AS host,
       rec.matches, rec.won, rec.drawn, rec.lost,
       rec.goals_for, rec.goals_against,
       q.performance                             AS stage_reached,
       s.position                                AS final_position, -- NULL outside
top four
       CASE s.position WHEN 1 THEN 'champion' WHEN 2 THEN 'runner-up'
                     WHEN 3 THEN 'third' WHEN 4 THEN 'fourth'
                     ELSE 'outside top four' END      AS finish,
       CASE WHEN s.position = 1 THEN TRUE ELSE FALSE END AS won_title
FROM host_countries h
JOIN tournaments tr      ON tr.tournament_id = h.tournament_id
JOIN teams t             ON t.team_id = h.team_id
JOIN qualified_teams q    ON q.tournament_id = h.tournament_id AND q.team_id = h.team_id
JOIN (SELECT tournament_id, team_id,
        COUNT(*)                                AS matches,
        SUM(CASE WHEN goals_for > goals_against THEN 1 ELSE 0 END) AS won,
        SUM(CASE WHEN goals_for = goals_against THEN 1 ELSE 0 END) AS drawn,
        SUM(CASE WHEN goals_for < goals_against THEN 1 ELSE 0 END) AS lost,
        SUM(goals_for)                                AS goals_for,
        SUM(goals_against)                             AS goals_against
FROM team_appearances
GROUP BY tournament_id, team_id) rec
LEFT JOIN tournament_standings s
        ON s.tournament_id = h.tournament_id AND s.team_id = h.team_id;

-- =====
-- V7. v_head_to_head - record of every team against every opponent

```

```

-- Purpose: One row per (team, opponent) pair that has met at a World Cup, from the first team's
point
-- of view: matches, wins, draws, losses, goals and first/last meeting. Filter by two team names to
-- get any head-to-head instantly (each pair appears twice, once from each side).
-- Based on: Q07 (head-to-head, self-join)
-- SQL concepts: JOIN of team_appearances to teams twice (team and opponent), GROUP BY on the pair,
-- conditional aggregation
-- Expected result: one row per ordered pair (1,230 rows = 615 pairs x 2); Brazil v Sweden: 7
matches,
-- 5 wins, 2 draws.
-- =====
CREATE OR REPLACE VIEW v_head_to_head AS
SELECT ta.team_id,
       t.team_name                               AS team,
       ta.opponent_id,
       o.team_name                               AS opponent,
       COUNT(*)                                  AS matches,
       SUM(CASE WHEN ta.goals_for > ta.goals_against THEN 1 ELSE 0 END) AS wins,
       SUM(CASE WHEN ta.goals_for = ta.goals_against THEN 1 ELSE 0 END) AS draws,
       SUM(CASE WHEN ta.goals_for < ta.goals_against THEN 1 ELSE 0 END) AS losses,
       SUM(ta.goals_for)                        AS goals_for,
       SUM(ta.goals_against)                    AS goals_against,
       MIN(tr.year)                             AS first_meeting,
       MAX(tr.year)                             AS last_meeting
FROM team_appearances ta
JOIN teams t          ON t.team_id = ta.team_id
JOIN teams o          ON o.team_id = ta.opponent_id
JOIN tournaments tr ON tr.tournament_id = ta.tournament_id
GROUP BY ta.team_id, t.team_name, ta.opponent_id, o.team_name;

```

---

*Data: Joshua C. Fjelstul, The Fjelstul World Cup Database, <https://github.com/jfjelstul/worldcup>, CC-BY-SA 4.0. No data values were modified. This document is a derived work under the same licence.*